



CDP 2026

September 2026

Project outcome

Goal: predict late/default payment while preserving chronological validation and operating a secure CPU-only service.

- Leakage-safe scikit-learn preparation pipeline.
- Explainable heuristic baseline, then reproducible TPE model search.
- Config-driven model packaging: family and hyperparameters change without API code changes.
- PostgreSQL-backed inference logging, drift monitoring, JWT roles, CI/CD and SAST.

Decision convention

Default class = **0**

Probabilities = [**P(default)**,
P(on-time)]

Training selection uses temporal validation only.

Git history: full branch graph (1/2)

```
* 5a3c003 (origin/master, origin/HEAD) Merge pull request #11 from 03loveslollipop/feat/repository-layout
|\
| * d4cb068 (HEAD -> feat/repository-layout, origin/feat/repository-layout) Reorganize pipeline and deployment layout
|/
* ca5ced1 (feat/cpu-xgboost-selection) Merge pull request #10 from 03loveslollipop/chore/showcase-monitoring-handoff
|\
| * 3b4a794 (origin/chore/showcase-monitoring-handoff, chore/showcase-monitoring-handoff) Add comprehensive project handoff
| * 7b77da0 Deploy Heroku services only from master
| * f213a6e Document monitoring showcase and enforce CPU deploys
|/
* 9abaaca (master) Merge pull request #9 from 03loveslollipop/feat/adaptive-training-search
|\
| * a9205c7 (origin/feat/adaptive-training-search) Merge origin/master into feat/adaptive-training-search
| |\
| |/\
|/|/
* | 77368c5 Merge pull request #8 from 03loveslollipop/feat/model-serving-monitoring
| |\
| |\ \
| |\ \ 6105457 Run CPU tests and publish SonarCloud coverage (#7)
| |\ \
| | | * b17d393 (feat/adaptive-training-search-clean) Keep tracked training CPU only
| | | * 5d91a86 Add CPU-tested adaptive model search
| | |/\
| | |/\
| | | * 7c30f1f (feat/adaptive-training-search) Clarify seeded sampling assertion
| | | * c264ad8 Keep CUDA experiments local only
| | | * 9189c9b Record adaptive integration results
```

The graph includes all reachable branches at deck generation time. Part 2 continues on the next slide.

Git history: full branch graph (2/2)

```

* fbc4c8d Close Optuna study connections
* 7e8bd1d Merge remote-tracking branch 'origin/chore/python-quality-gates' into feat/adaptive-training-search
//
//
//
* b577672 Add budgeted TPE model search
//
//
//
* 96dd7e8 (origin/feat/model-serving-monitoring, feat/model-serving-monitoring) Cover serving boundaries and migration paths
* db056ee Merge remote-tracking branch 'origin/chore/python-quality-gates' into feat/model-serving-monitoring
//
//
//
* dee9cb4 Harden service binds and artifact paths
* 1004481 Refresh serving checklist and remove duplicate image
* a58a973 (origin/chore/python-quality-gates, chore/python-quality-gates) Install serving dependencies when present
* 5920b77 Pin Ruff to validated version
* 4cf87f9 Merge remote-tracking branch 'origin/master' into feat/model-serving-monitoring
//
//
//
* 8324dc0 Add CPU tests and coverage to SonarCloud
//
//
* d5a4649 Merge pull request #6 from 03loveslollipop/chore/sonarcloud-actions
//
//
* 567cab2 (origin/chore/sonarcloud-actions, chore/sonarcloud-actions) Add SonarCloud analysis workflow
//
//
* 4763513 Split serving into independent services
* 1c2c1ee Merge pull request #5 from 03loveslollipop/feat/model_training_evaluation
//
//
//
* ad472ca Record database auth deployment
* 858b66a Persist authentication users in PostgreSQL
* cf5e879 Add JMT roles and visual inference
* cb7b659 Configure daily Heroku monitoring
* b60eeaf Preserve model identity across rollouts
* 9c38886 Update GitHub Actions runtimes
* 037feb2a Deploy model serving and monitoring
//
//
//
* 1ea00d4 (origin/feat/model-training-notebook, feat/model-training-notebook) Add model training notebook
//
//
* bc41ac1 (feat/model training evaluation) Add model training pipeline and optimization handoff
* 5e8593d Merge pull request #2 from 03loveslollipop/feat/feat_engineering_pipeline
//
//
//
* 6508274 Add EDA-based heuristic classifier
//
//
//
* 2a1d94e Add chronological train test split
* 45f0d70 Harden predictive data preparation
* f0cfadc Add contribution guidelines
* e66dd74 Add credit feature engineering pipeline
//
//
//
* fbb6fa7 Remove ResultsPresentation.pptx and update ResultsReport.pdf
* 515c62 Update README, report, figures, and EDA notebook
* 526cdfb Rewrite README as a summary of the findings, with figures
* dc88417 Version the source dataset alongside the code
* 1183653 Add results report and presentation, document the pipeline decisions
* 17e5797 Remove .gitkeep and add eda.ipynb for exploratory data analysis
* 3fe0ec9 Add .gitkeep to etl scripts/src/development directory

```

The highlighted review sequence and screenshot placeholders are shown next.

Reviewed pull requests

PR #2

Feature-engineering pipeline

PR #2 screenshot

Replace with GitHub PR
and review screenshot

PR #3

Heuristic baseline into feature branch

PR #3 screenshot

Replace with GitHub PR
and review screenshot

PR #4

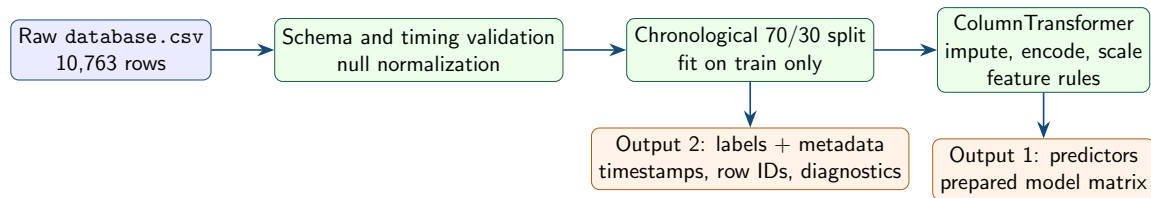
Review screenshot requested

PR #4 screenshot

Replace with GitHub PR
and review screenshot

Note: #2 and #3 are merged PRs. GitHub has no resolvable #4 in this repository at generation time; this is intentionally a replaceable placeholder rather than fabricated evidence.

EDA and scikit-learn preparation flow



Excluded: target, bureau score leakage and metadata. Missingness indicators and validation flags are retained; no rows are silently removed.

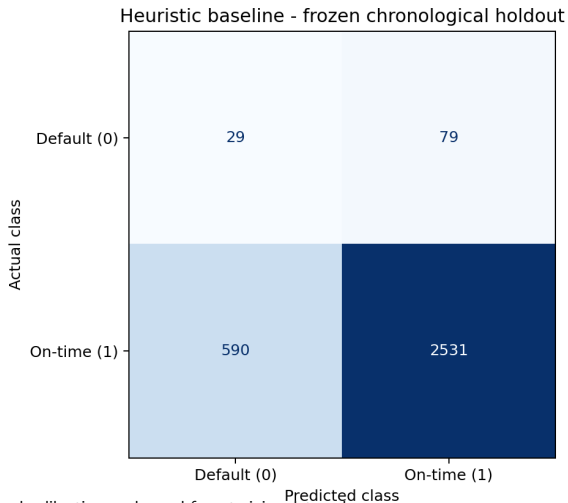
Baseline: explainable heuristic policy

What it does

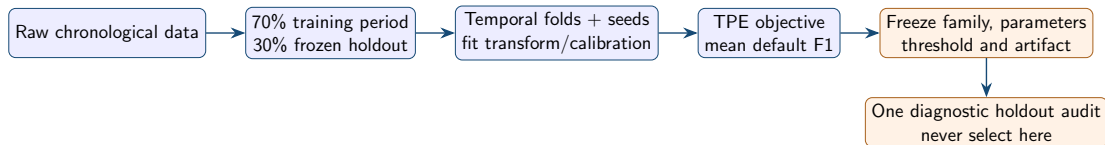
Combines EDA-derived credit signals into a transparent rule score. It is a reference policy, not a search candidate, so it anchors improvement claims.

Frozen holdout metric	Heuristic
Default precision	0.0468
Default recall	0.2685
Default F1	0.0798
Average precision	0.0858
ROC-AUC	0.6241

Frozen chronological holdout: 3,229 rows; default prevalence 3.34%. Thresholds and calibration are learned from training data only.



Training protocol: prevent leakage before optimizing



- Default-class F1 is primary; temporal/seed stability and CPU cost resolve close results.
- Calibration and threshold selection use out-of-fold training scores, not holdout labels.

Models tried and TPE search space

Learned families

- Logistic regression, decision tree, Gaussian NB, RBF SVM.
- Random forest and Extra Trees.
- XGBoost, LightGBM, PyTorch MLP.

Reference policies

- EDA heuristic and Dummy classifier.

TPE controls

- 30 trials per learned family, serial/resumable SQLite study.
- Reproducible seed and training-only protocol fingerprint.
- Trees: depth/leaves, estimators, learning rate, sampling, regularization, class weights.
- MLP: widths, learning rate, dropout, batch size, epochs, weight decay.

CPU TPE rerun: strongest candidates

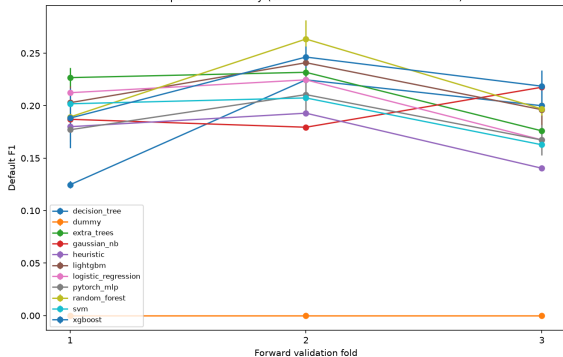
Model	F1	temporal sd	AP	ROC-AUC	CPU batch ms
Random forest	0.2166	0.0333	0.1918	0.6987	106.0
XGBoost	0.2177	0.0238	0.2016	0.7018	44.6
LightGBM	0.2134	0.0197	0.1823	0.6897	52.0
Heuristic	0.1711	0.0223	0.1443	0.6620	52.4

Interpretation

XGBoost leads F1, AP, ROC-AUC and CPU latency. LightGBM has lower temporal variation. The deployed contract remains the validated XGBoost configuration; the latest selector's stability weighting is a documented governance decision to revisit, not a holdout-driven model swap.

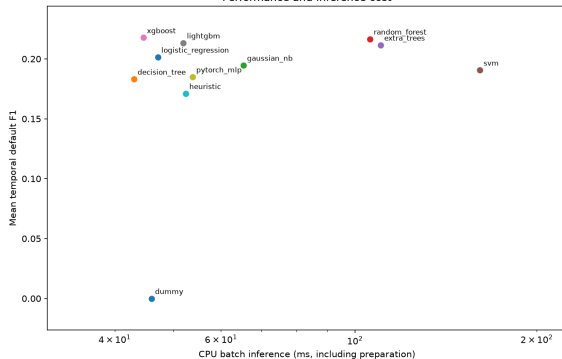
Model comparison visuals

Temporal consistency (error bars: seed standard deviation)



Temporal-validation F1

Performance and inference cost



F1 versus CPU inference cost

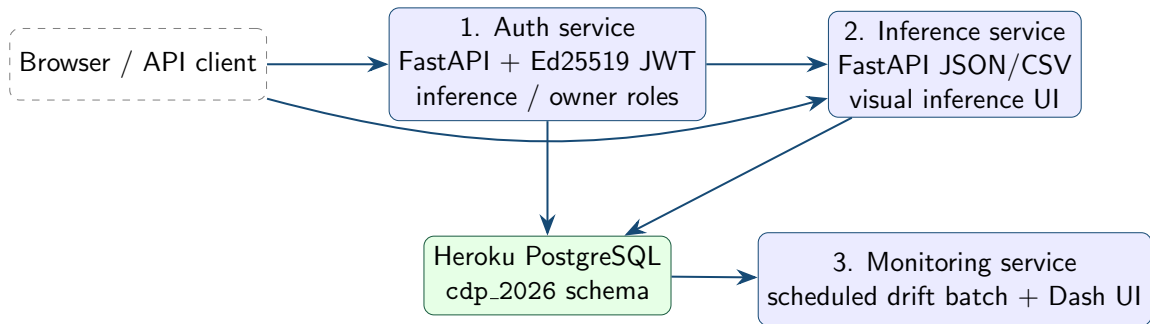
Selected deployment contract: XGBoost

Config-driven release

`deployment_model_config.json` contains the family, fixed hyperparameters, CPU-only runtime, class order and batch limit. The release workflow trains the configured family and packages only its runtime dependency.

Parameter	Value
estimators	1,100
max depth	5
learning rate	0.00818
min child weight	1.645
subsample / columns	0.672 / 0.546
L1 / L2 regularization	0.363 / 0.049
runtime	CPU, class order [0,1]

Production deployment: three logical microservices



Stack: Python, FastAPI, Dash, SQLAlchemy, PostgreSQL 17, Docker, Heroku Container Registry, GitHub Actions and SonarCloud. Monitoring is one logical service deployed as separate batch and UI processes for operational isolation.

Drift-monitoring showcase result

Forced seven-day window

- 120 deliberately shifted synthetic rows sent through normal inference validation, idempotency and PostgreSQL logging.
- 121 prediction rows analyzed after catch-up processing.
- 46 aggregate monitoring metrics: **22 ok, 19 alert, 4 warning, 1 insufficient data.**
- Mean default probability: 0.075477; all 120 showcase decisions were on-time.

Correct interpretation

The shifted batch demonstrates that feature/score drift and data-quality alerts work. It is **not** model-validation evidence. Performance/calibration stay “insufficient data” until mature outcomes arrive.

SAST findings: context and remediation

S8544 - dependency locking

- Service requirement files use exact validated versions (for example FastAPI, SQLAlchemy, XGBoost CPU).
- The screenshot is historical; dependency versions are now pinned per service image.
- Remaining improvement: add hash-locked generated constraints when supply-chain policy requires artifact integrity verification.

S8707 - path traversal through CLI path

- This was remediated, not dismissed as a false positive.
- The installer no longer accepts a file path from `sys.argv`; it reads only its co-located, repository-owned deployment contract.
- The trusted CI environment reduces exposure, but removing unnecessary input is the durable control.

Evidence to add before delivery

Insert the SonarCloud screenshots: resolved S8544 and closed S8707 finding.

Takeaways and next steps

- The pipeline is reproducible, CPU-deployable and separated from local CUDA experimentation.
- Production captures inference events safely and calculates aggregate drift/performance with explicit insufficient-data states.
- The branch and PR history makes engineering review auditable from EDA through production.
- Next: settle the explicit XGBoost-versus-LightGBM tie-break policy, collect matured outcomes, and replace deck placeholders with review/Sonar screenshots.

Questions